

第10章 排序

Faculty of Information and Control Engineering

Huanliang Sun

Email: sunhl@sjzu.edu.cn

Office: Wu(戊2-306) 24690666

第十章 内部排序

第一次课

10.1 概述

10.2 插入排序

练习与作业

第二次课

10.3 快速排序

10.4 选择排序

练习与作业

第三次课

10.5 归并排序

10.6 基数排序

排序的定义： 将一个数据元素（或记录）的任意序列，重新排列成一个按关键字有序的序列。

给定关键字序列

{ 52, 49, 80, 36, 14, 58, 61, 23, 97, 75 }

调整为

{ 14, 23, 36, 49, 52, 58, 61, 75, 80, 97 }

排序的稳定性： 若相同关键字元素间的位置关系，排序前与排序后保持一致，则此排序方法是**稳定的**；而不能保持一致的排序方法称为**不稳定的**。

排序基本操作：

- 比较两个关键字大小
- 将记录从一个位置移动到另一个位置

□ 排序分类

➤ 按待排序记录所在位置

- 内部排序：待排序记录存放在内存
- 外部排序：排序过程中需对外存进行访问的排序

➤ 按排序依据原则

- 插入排序：直接插入排序、折半插入排序、希尔排序
- 交换排序：冒泡排序、快速排序
- 选择排序：简单选择排序、堆排序
- 归并排序：2-路归并排序
- 基数排序

➤ 按排序所需工作量

- 简单的排序方法： $T(n)=O(n^2)$
- 先进的排序方法： $T(n)=O(n\log_2 n)$
- 基数排序： $T(n)=O(d.n)$

➤ 按是否改变结点的物理位置分类：

- 改变结点的物理位置
- 不改变结点的物理位置
 - 链地址法（地址排序）：即对链表进行排序
 - 利用辅助地址表进行的排序：静态链表的排序
 - 计数排序：通过比较，计算序列中有多少个结点的关键字比他小，以此决定逻辑顺序

本章约定的数据类型 p264

返回

一、直接插入排序 最简单的一种排序

排序过程：整个排序过程为 $n-1$ 趟插入，即先将序列中第1个记录看成是一个有序子序列，然后从第2个记录开始，逐个进行插入，直至整个序列有序。

算法描述：实现一趟插入排序的步骤 $R[0]$ 作监视哨

- 1) 在 $R[1..i-1]$ 中查找 $R[i]$ 的插入位置，即确定 $j(0 \leq j < i)$ 使得 $R[1..j].key \leq R[i].key < R[j+1..i-1].key$
- 2) 将 $R[j+1..i-1]$ 中的记录后移一个位置；
- 3) 将 $R[i]$ 插入到 $j+1$ 的位置。

```

void InsertSort ( SqList &L) {
// 对顺序表L作直接插入排序
    for ( i=2; i<=L.length; ++i )
        if ( L.r[i].key < L.r[i-1].key ) {
            // 将 L.r[i] 插入有序子表
            L.r[0] = L.r[i]; L.r[i]=L.r[i-1];
            for ( j=i-2; L.r[0].key < L.r[j].key; --j )
                L.r[j+1] = L.r[j];           // 记录后移
            L.r[j+1] = L.r[0];           // 插入到正确位置
        } // if
    } // InsertSort

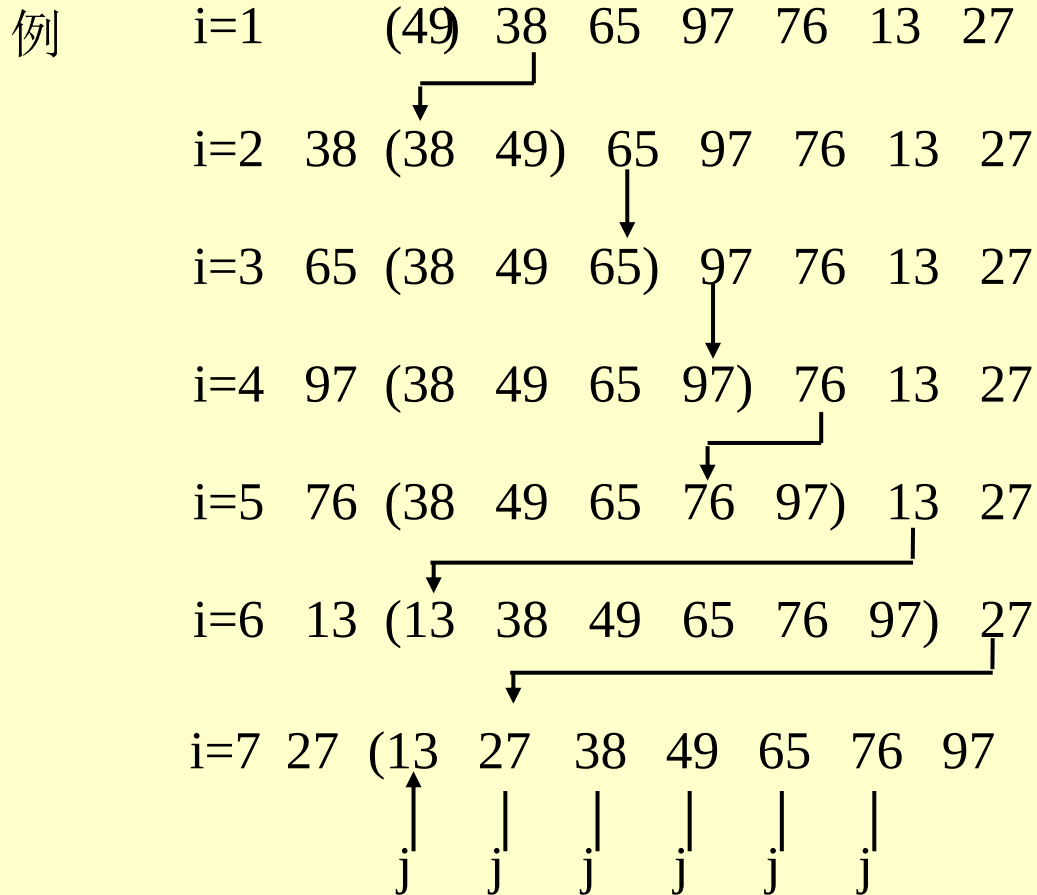
```

```

void InsertSort (ElemType A[], int n) {
    for ( i=2; i<=n; ++i ) //依次将A[2]- A[n]插入到前面已排序序列
        if ( A[i] < A[i-1] ) {
            A[0] = A[i]; //复制为哨兵， A[0]不存放元素
            for ( j=i-1; A[0] < A[j]; --j )
                A[j+1] = A[j];           // 记录后移
            A[j+1] = A[0];           // 插入到正确位置
        }
    }
}

```





排序结果:(13 27 38 49 65 76 97)

最后一趟开始前所有元素都可能不在最终位置上。



❖ 算法评价

⌘ 时间复杂度

❖ 若待排序记录按关键字从小到大排列(正序)

☆ 关键字比较次数: $\sum_{i=2}^n 1 = n - 1$

☆ 记录移动次数: 0

❖ 若待排序记录按关键字从大到小排列(逆序)

☆ 关键字比较次数: $\sum_{i=2}^n i = \frac{(n+2)(n-1)}{2}$

☆ 记录移动次数: $\sum_{i=2}^n (i+1) = \frac{(n+4)(n-1)}{2}$

❖ 若待排序记录是随机的, 取平均值

☆ 关键字比较次数: $\frac{n^2}{4}$

☆ 记录移动次数: $\frac{n^2}{4}$

$T(n) = O(n^2)$

⌘ 空间复杂度: $S(n) = O(1)$

二、其他插入排序

1、折半插入排序

排序过程：用折半查找方法确定插入位置的排序

i=1		(30)	13	70	85	39	42	6	20
i=2	13	(13	30)	70	85	39	42	6	20
⋮									
i=7	6	(6	13	30	39	42	70	85)	20
i=8	20	(6	13	30	39	42	70	85)	20
		↑			↑			↑	
		s			m			j	
i=8	20	(6	13	30	39	42	70	85)	20
		↑	↑	↑					
		s	m	j					
i=8	20	(6	13	30	39	42	70	85)	20
			↑	↑	↑				
			j	s	m				
i=8	20	(6	13	20	30	39	42	70	85)

- 算法描述

```
void BInsertSort (SqList &L) {  
    // 对顺序表L作折半插入排序  
    for ( i=2; i<=L.length; ++i ){  
        L.r[0] = L.r[i];      // 将L.r[i]暂存到L.r[0]  
        low = 1; high = i-1;  
        while (low<=high){  
            // 在r[low..high]中折半查找有序插入的位置  
            m = (low+high)/2;      // 折半  
            if (L.r[0].key < L.r[m].key) high = m-1;  
            // 插入点在低半区  
            else low = m+1;      // 插入点在高半区  
        } // while  
        for ( j=i-1; j>=low; --j ) L.r[j+1] = L.r[j]; // 记录后移  
        L.r[high+1] = L.r[0];      // 插入  
    }  
} // BInsertSort
```

- ❖ 算法评价

- ❖ 时间复杂度: $T(n)=O(n^2)$
- ❖ 空间复杂度: $S(n)=O(1)$

```
void InsertSort (ElemType A[], int n) {  
    int i, j, low, high, mid;  
    for ( i=2; i<=n; ++i ) //依次将A[2]- A[n]插入到前面已排序序列  
        if ( A[i] < A[i-1] ) {  
            A[0] = A[i]; //复制为哨兵， A[0]不存放元素  
            low=1; high=i-1;  
            while(low<=high){  
                mid=(low+high)/2;  
                if(A[mid]>A[0]) high=mid-1;  
                else low=mid+1;  
            }  
            for ( j=i-1; j>=high+1; --j )  
                A[j+1] = A[j];           // 记录后移  
            A[high+1] = A[0];           // 插入到正确位置  
        }  
}
```

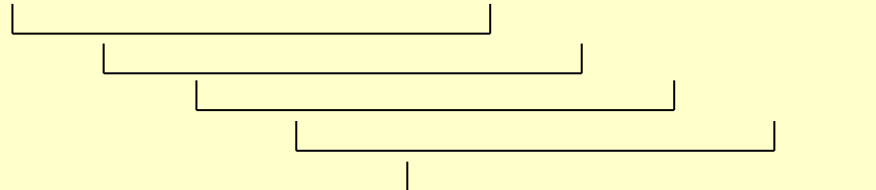
2、希尔排序(缩小增量法)

基本思想：先取一个正整数 $d_1 < n$ ，把所有相隔 d_1 的记录放一组，组内进行直接插入排序；然后取 $d_2 < d_1$ ，重复上述分组和排序操作；直至 $d_i = 1$ ，即所有记录放进一个组中排序为止。

例 初始: 49 38 65 97 76 13 27 48 55 4

取 $d_1=5$

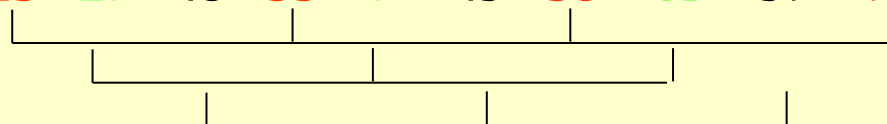
一趟分组: 49 38 65 97 76 13 27 48 55 4



一趟排序: 13 27 48 55 4 49 38 65 97 76

取 $d_2=3$

二趟分组: 13 27 48 55 4 49 38 65 97 76



二趟排序: 13 4 48 38 27 49 55 65 97 76

取 $d_3=1$

三趟分组: 13 4 48 38 27 49 55 65 97 76

三趟排序: 4 13 27 38 48 49 55 65 76 97

算法描述:

```
void ShellInsert ( SqList &L, int dk) {  
    // 对顺序表L作一趟增量为dk的希尔排序  
    for ( i=dk+1; i<=L.length; ++i )  
        if ( L.r[i].key < L.r[i-dk].key ) { // 将L.r[i]插入有序子表  
            L.r[0] = L.r[i]; L.r[i]=L.r[i-dk];  
            for ( j=i-2*dk; j>0 && L.r[0].key < L.r[j].key; j-=dk )  
                L.r[j+dk] = L.r[j]; // 记录后移  
            L.r[j+dk] = L.r[0]; // 插入到正确位置  
        } // if  
    } // ShellInsert  
void ShellSort (SqList &L, int dlta[], int t){  
    // 按增量序列 dlta[0..t-1] 对顺序表L作希尔排序  
    for (k=0; k<t; ++t)  
        ShellInsert(L, dlta[k]);  
        // 一趟增量为 dlta[k] 的插入排序  
    } // ShellSort
```

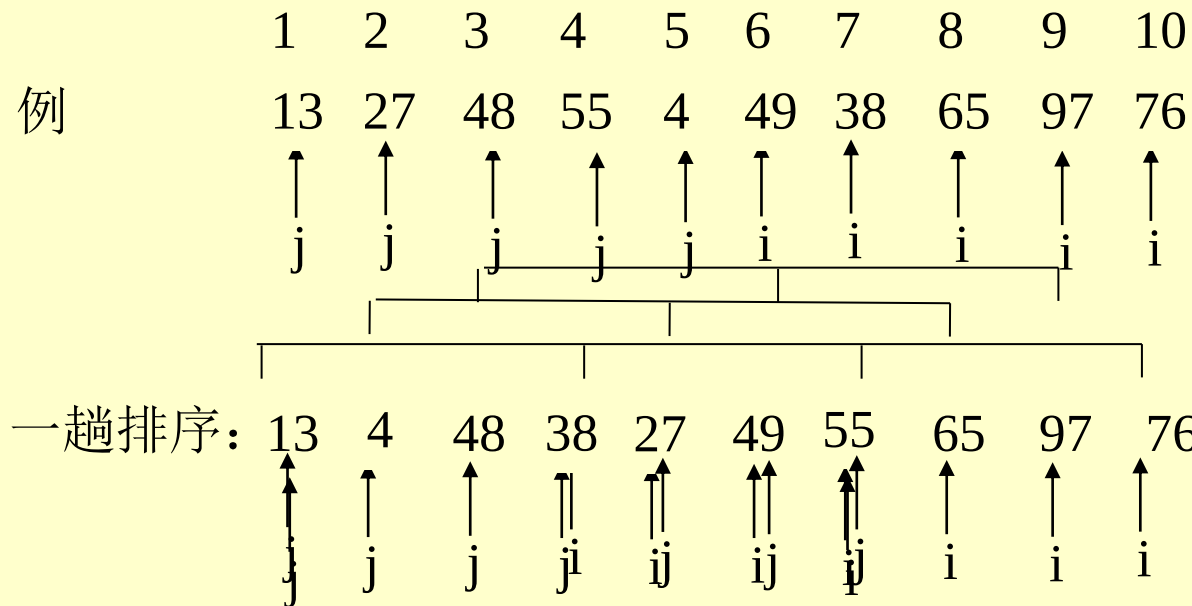


```

void ShellSort (ElemType A[], int n) {
//A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已
    for ( dk=n/2; dk>=1; dk=dk/2 ) //步长变化
        for ( i=dk+1; i<=n; ++i)
            if ( A[i] < A[i-dk] ) {
                A[0] = A[i]; //暂存在A[0]
                for ( j=i-dk; j>0&& A[0] < A[j]; j-=dk )
                    A[j+dk] = A[j]; // 记录后移
                A[j+dk] = A[0]; // 插入到正确位置
            }
}

```

```
#define T 3
int d[]={5,3,1};
```



一趟排序:

二趟排序: 13 4 48 38 27 49 55 65 97 76



❧ 希尔排序特点

- ❖ 子序列的构成不是简单的“逐段分割”，而是将相隔某个增量的记录组成一个子序列
- ❖ 希尔排序可提高排序速度，因为
 - ❧ 分组后 n 值减小， n^2 更小，而 $T(n)=O(n^2)$ ，所以 $T(n)$ 从总体上看是减小了
 - ❧ 关键字较小的记录跳跃式前移，在进行最后一趟增量为1的插入排序时，序列已基本有序
- ❖ 增量序列取法
 - ❧ 无除1以外的公因子
 - ❧ 最后一个增量值必须为1

练习

- 1、给出关键字序列{4,5,1,2,6,3}的直接插入排序过程。
- 2、给出关键字序列{50,26,38,80,70,90,8,30,40,20}的希尔排序过程，增量序列为 $d=\{5,3,1\}$,排序结果从小到大排列。

18、【2012统考真题】对同一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是（）。

A、排序的总趟数 B、元素的移动次数 C、使用辅助空间的数量 D、元素之间的比较次数

19、数据序列{8,10,13,4,6,7,22,2,3}只能是（）两趟排序后的结果。

A、简单选择排序 B、起泡排序 C、直接插入排序 D、堆排序

20、对序列{15,9,7,8,20,-1,4}用希尔排序方法排序，经一趟后序列变为{15,-1,4,8,20,9,7}，则该次采用的增量是（）。

A、1 B、2 C、3 D、4

21、【2014统考真题】用希尔排序方法对一个数据序列进行排序时，若第1趟排序结果为9,1,4,13,7,8,20,23,15,则该趟排序采用的增量可能是（）。

A、2 B、3 C、4 D、5

23、以下排序算法中，不稳定的是（）。

A、冒泡排序 B、直接插入排序 C、希尔排序 D、归并排序

24、以下排序算法中，稳定的是（）。

A、快速排序 B、堆排序 C、直接插入排序 D、简单选择排序

25、【2015统考真题】希尔排序的组内排序采用的是（）。

A、直接插入排序 B、折半插入排序 C、快速排序 D、归并排序

26、【2018统考真题】对初始数据序列（8,3,9,11,2,1,4,7,5,10,6）进行希尔排序。若第一趟排序结果为(1,3,7,5,2,6,4,9,11,10,8)，第二趟排序结果为(1,2,6,4,3,7,5,8,11,10,9)，则两趟排序采用的增量依次是()

A、3,1 B、3,2 C、5,2 D、5,3